Build-Systeme: Apache Ant

Carsten Gips (HSBI)

TL;DR
Zum Automatisieren von Arbeitsabläufen (Kompilieren, Testen, …) stehen in der Java-Welt verschiedene Tools zur Verfügung: Apache Ant, Apache Maven und Gradle sind sicher die am bekanntesten darunter.
In Apache Ant werden die Build-Skripte in XML definiert. Die äußere Klammer ist dabei das `<project>`. In einem Projekt kann es ein oder mehrere Teilziele (*Targets*) geben, die untereinander abhängig sein können. Die Targets können quasi "aufgerufen" werden bzw. in der IDE selektiert und gestartet werden.
In einem Target kann man schließlich mit *Tasks* Aufgaben wie Kompilieren, Testen, Aufräumen, … erledigen lassen. Dazu gibt es eine breite Palette an vordefinierten Tasks. Zusätzlich sind umfangreiche Operationen auf dem Filesystem möglich (Ordner erstellen, löschen, Dinge kopieren, …).
Über *Properties* können Werte und Namen definiert werden, etwa für bestimmte Ordner. Die Properties sind unveränderliche Variablen (auch wenn man sie im Skript scheinbar neu setzen kann).
Über Apache Ivy können analog zu Maven und Gradle definierte Abhängigkeiten aus Maven-Central aufgelöst werden.
Im Unterschied zu Maven und Gradle ist in Ant *kein* Java-Entwicklungsmodell eingebaut. Man muss sämtliche Targets selbst definieren.
Videos

- VL Apache Ant
- Demo Aufruf von Ant (Konsole, IDE: hello.xml)
- Demo Properties, Targets, Dependencies (build.xml)
- Demo Abhängigkeiten mit Ivy auflösen (ivydemo.xml)

---

1 Automatisieren von Arbeitsabläufen

Works on my machine …

- Build-Tools:
  - **Apache Ant**
  - Apache Maven
  - Gradle

- Aufgaben:
  - Übersetzen des Quellcodes
  - Ausführen der Unit-Tests
  - Generieren der Dokumentation
  - Packen der Distribution
  - Aufräumen temporärer Dateien
  - …

=> Automatisieren mit Apache Ant: ant.apache.org

2 Aufbau von Ant-Skripten: Projekte und Targets

```
<project name="Vorlesung" default="clean" basedir=".">
    <target name="init" />
    <target name="compile" depends="init" />
    <target name="test" depends="compile" />
    <target name="dist" depends="compile,test" />
    <target name="clean" />
</project>
```

- Ein Hauptziel: `project`
- Ein oder mehrere Teilziele: `target`
  - Abhängigkeiten der Teilziele untereinander möglich: `depends`
  - Vorbedingung für Targets mit **`if`** oder `unless`

3 Aufgaben erledigen: Tasks

- **Tasks**: Aufgaben bzw. Befehle ("Methodenaufruf"), in Targets auszuführen
- Struktur: `<taskname attr1="value1"attr2="value2"/>`

```
<target name="simplecompile" depends="clean,init">
    <javac srcdir="src" destdir="build" classpath="." />
</target>
```

- Beispiele: `echo`, `javac`, `jar`, `javadoc`, `junit`, ...
- Je Target mehrere Tasks möglich
- Quellen:
  - Eingebaute Tasks
  - Optionale Task-Bibliotheken
  - Selbst definierte Tasks

=> Überblick: ant.apache.org/manual/tasksoverview.html

Konsole/IDE: ant -f hello.xml

4 Properties: Name-Wert-Paare

```
<property name="app"       value="MyProject" />
<property name="build.dir" location="build" />

<target name="init">
    <mkdir dir="${build.dir}" />
</target>
```

- Setzen von Eigenschaften: `<property name="wuppie"value="fluppie"/>`
  - Properties lassen sich nur einmal setzen (“immutable”)
    - Erneute Definition ist wirkungslos, erzeugt aber leider keinen Fehler
- Nutzung von Properties: `<property name="db"value="${wuppie}.db"/>`
- Pfade: “`location`” statt “`value`” nutzen: `<property name="ziel"location="${p}/bla/blub"/>`

- Properties beim Aufruf setzen mit Option “`-D`”:
  `ant -Dwuppie=fluppie`

Beispiel build.xml, Properties

5 Tasks zum Umgang mit Dateien und Ordnern

```
<target name="demo">
    <mkdir dir="${build.dir}/lib" />

    <delete dir="${build.dir}" />
    <delete file="${dist.dir}/wuppie.jar" />

    <copy file="myfile.txt" tofile="../bak/mycopy.txt" />
    <move file="src/file.orig" tofile="bak/file.moved" />
</target>
```

- `<mkdir dir="${dist}/lib"/>`
  - Legt auch direkte Unterverzeichnisse an
  - Keine Aktion, falls Verzeichnis existiert
- `<delete dir="${builddir}"/>`
  - Löscht eine Datei (“`file`”) oder ein Verzeichnis (“`dir`”) (rekursiv!)
- `<copy file="myfile.txt"tofile="../bak/mycopy.txt"/>`
- `<move file="src/file.orig"tofile="bak/file.moved"/>`

Beispiel build.xml, init und clean

6 Nutzung von Filesets in Tasks

```
<copy todir="archive">
    <fileset dir="src">
        <include name="**/*.java" />
        <exclude name="**/*.ba?" />
    </fileset>
</copy>
```

```

<delete>
    <fileset dir="." includes="**/*.ba?" />
</delete>
```

- “*” für beliebig viele Zeichen
- “?” für genau ein Zeichen
- “**” alle Unterverzeichnisse

Es gibt auch die Variante `<dirset dir="...">`, um Verzeichnisse zu gruppieren.

7 Pfade und externe Bibliotheken

- Als Element direkt im Task:

```
<classpath>
    <pathelement location="${lib}/helper.jar" />
    <pathelement path="${project.classpath}" />
</classpath>
```

D.h. die Einbettung in den `javac`-Task würde etwa so erfolgen:

```
<target ... >
    <javac ...>
        <classpath>
            <pathelement location="${lib}/helper.jar" />
            <pathelement path="${project.classpath}" />
        </classpath>
    </javac>
</target>
```

Anmerkung: Neben dem `pathelement` können Sie hier auch (wie im nächsten Beispiel gezeigt) ein oder mehrere `fileset` nutzen.

- Wiederverwendbar durch ID und “`refid`”:

```
<path id="java.class.path">
    <fileset dir="${lib}">
        <include name="**/*.jar" />
    </fileset>
</path>

<classpath refid="java.class.path" />
```

Die Einbettung in den `javac`-Task würde hier etwa so erfolgen:

```
<path id="java.class.path">
    <fileset dir="${lib}">
        <include name="**/*.jar" />
    </fileset>
```

```
5  </path>
6
7  <target ... >
8      <javac ...>
9          <classpath refid="java.class.path" />
10     </javac>
11 </target>
```

Anmerkung: Neben dem `fileset` können Sie hier auch (wie oben gezeigt) ein oder mehrere `pathelement` nutzen.

Anmerkung: Laut ant.apache.org/manual/Tasks/junit.html benötigt man neben der aktuellen `junit.jar` noch die `ant-junit.jar` im Classpath, um mit dem `junit`-Ant-Task entsprechende JUnit4-Testfälle ausführen zu können.

Für JUnit5 gibt es einen neuen Task `JUnitLauncher` (vgl. ant.apache.org/manual/Tasks/junitlauncher.html).

8 Beispiele

8.1 Beispiel-Task: Kompilieren

```
1 <path id="project.classpath">
2     <fileset dir="${lib.dir}" includes="**/*.jar" />
3 </path>
4
5 <target name="compile" depends="init" description="compile the source " >
6     <javac srcdir="${src.dir}" destdir="${build.dir}">
7         <classpath refid="project.classpath" />
8     </javac>
9 </target>
```

8.2 Beispiel-Task: Packen

```
1 <target name="dist" depends="compile" description="generate the distribution" >
2     <mkdir dir="${dist.dir}" />
3     <jar jarfile="${dist.dir}/${app}.jar" basedir="${build.dir}">
4         <manifest>
5             <attribute name="Main-Class"  value="${app}" />
6         </manifest>
7     </jar>
8 </target>
```

8.3 Beispiel-Task: Testen

- Tests einer Testklasse ausführen:

```
<junit>
    <test name="my.test.TestCase" />
</junit>
```

- Test ausführen und XML mit Ergebnissen erzeugen:

```
<junit printsummary="yes" fork="yes" haltonfailure="yes">
    <formatter type="xml" />
    <test name="my.test.TestCase" />
</junit>
```

- Verschiedene Tests als Batch ausführen:

```
<junit printsummary="yes" haltonfailure="yes">
    <classpath>
        <pathelement location="${build.tests}" />
        <pathelement path="${java.class.path}" />
    </classpath>

    <formatter type="plain"/>

    <test name="my.test.TestCase" haltonfailure="no" outfile="result">
        <formatter type="xml" />
    </test>

    <batchtest fork="yes" todir="${reports.tests}">
        <fileset dir="${src.tests}">
            <include name="**/*Test*.java" />
            <exclude name="**/AllTests.java" />
        </fileset>
    </batchtest>
</junit>
```

- Aus Testergebnis (XML) einen Report generieren:

```
<junitreport todir="${reportdir}">
    <fileset dir="...">
        <include name="TEST-*.xml" />
    </fileset>
    <report format="frames" todir="..." />
</junitreport>
```

- Abbruch bei Fehler:

```
<junit ... failureproperty="tests.failed" ... >
    <fail if="tests.failed" />
</junit>
```

=> `junit.jar` und `ant-junit.jar` (JUnit4.x) im Pfad!

8.4 Programme ausführen

```
<target name="run">
```

```
    <java jar="build/jar/HelloWorld.jar" fork="true" classname ="test.Main">
        <arg value ="-h" />
        <classpath>
            <pathelement location="./lib/test.jar" />
        </ classpath>
    </ java>
</target>
```

9 Ausblick: Laden von Abhängigkeiten mit Apache Ivy

Apache Ivy: Dependency Manager für Ant

```
<!-- build.xml -->
<project xmlns:ivy="antlib:org.apache.ivy.ant">
    <target name="resolve">
        <ivy:retrieve/>
    </target>
</project>
```

Wenn Ivy installiert ist, kann man durch den Eintrag `xmlns:ivy="antlib:org.apache.ivy.ant"` in der Projekt-Deklaration im Ant-Skript die Ivy-Tasks laden. Der wichtigste Task ist dabei `ivy:retrieve`, mit dem externe Projektabhängigkeiten heruntergeladen werden können.

```
<!-- ivy.xml -->
<ivy-module version="2.0">
    <dependencies>
        <dependency org="commons-cli" name="commons-cli" rev="1.5.0" />
        <dependency org="junit" name="junit" rev="4.13.2" />
    </dependencies>
</ivy-module>
```

Zur Steuerung von Ivy legt man eine weitere Datei `ivy.xml` an. Das Wurzelelement ist `ivy-module`, wobei die `version` die niedrigste kompatible Ivy-Version angibt.

Der `dependencies`-Abschnitt definiert dann die Abhängigkeiten, die Ivy auflösen muss. Die Schreibweise ist dabei wie im Maven2 Repository (mvnrepository.com) angelegt. Dort findet man beispielsweise für Apache Commons CLI den Eintrag für Maven (“POM”-Datei):

```
<!-- https://mvnrepository.com/artifact/commons-cli/commons-cli -->
<dependency>
    <groupId>commons-cli</groupId>
    <artifactId>commons-cli</artifactId>
    <version>1.4</version>
</dependency>
```

Für die Ivy-Konfiguration übernimmt man die `groupId` als `org`, die `artifactId` als `name` und die `version` als `rev` im Eintrag `dependency`.

Damit kann Ivy diese Bibliothek über den Ant-Task `ivy:retrieve` vor dem Bauen herunterladen, sofern die Bibliothek noch nicht lokal vorhanden ist. Eventuelle Abhängigkeiten werden dabei ebenfalls aufgelöst.

Im Detail: Der Ant-Task `ivy:retrieve` löst zunächst die Abhängigkeiten auf und lädt die Dateien (sofern sie noch nicht vorhanden oder veraltet sind) in den Ivy-Cache (per Default: `~/.ivy2/cache/`). Danach werden die Dateien in den Default-Library-Order im Projekt kopiert (per Defaul: `./lib/`). Die Ordner kann man über Optionen im `ivy:retrieve`-Task einstellen.

Demo: ivydemo.xml

10 Ausblick: Weitere Build-Systeme

- Maven
  - War als Nachfolger von Ant gedacht
  - Statt wie bei Ant explizit Targets zu formulieren, geht Maven von einem Standardprojekt aus - nur noch Abweichungen müssen formuliert werden
  - Zieht Abhängigkeiten in zentralen `.maven`-Ordner

- Gradle
  - Eine Art Mischung aus Ant und Maven unter Nutzung der Sprache Groovy

- Make
  - DER Klassiker, stammt aus der C-Welt. Kann aber natürlich auch Java.
  - Analog zu Ant: Aktionen und Ziele müssen explizit definiert werden

11 Wrap-Up

Apache Ant: ant.apache.org

- Automatisieren von Arbeitsabläufen
- Apache Ant: Targets, Tasks, Properties
  - Targets sind auswählbare Teilziele
  - Abhängigkeiten zwischen Targets möglich
  - Tasks erledigen Aufgaben (innerhalb Targets)
  - Properties sind nicht änderbare Variablen
  - Umfangreiche Operationen auf Filesystem möglich

12 Zum Nachlesen

- Inden (2013, Abschn. 2.5.2)

Lernziele

- k3: Schreiben einfacher Ant-Skripte mit Abhängigkeiten zwischen den Targets
- k3: Nutzung von Ant-Filesets (Dateisystemoperationen, Classpath)
- k3: Nutzung von Ant-Properties
- k3: Ausführen von Ant-Targets aus der IDE heraus

Quizzes

- Quiz Apache Ant (ILIAS)

Quellen

Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9289c1f (Remove Hugo: remove 'cbox', 2025-04-29)